

Week 1 Reflection: Multi-Agent Research & Report Crew

Course: UCLA Extension — Agentic AI

Author: Bhavesh Upadhyaya

Repository: github.com/bhaveshu/agents-assignment-1

1. Architectural & Agent Design: The "Why"

For this assignment, I implemented a 4-agent sequential workflow in CrewAI designed to produce comprehensive academic literature reviews from a curated 15-paper corpus. The core architectural philosophy was **strict separation of concerns** across four specialized personas:

1. **Query Expander (Research Query Strategist):** Focuses solely on conceptual problem decomposition. It translates ambiguous, high-level research questions into 3–5 targeted sub-questions, academic keywords, and search queries.
2. **Source Hunter (Academic Literature Scout):** The dedicated RAG specialist. It is the **only agent equipped with tools** (`search_papers`). By isolating tool execution to this single agent, the pipeline avoids tool-calling confusion and hallucinations in downstream reasoning agents.
3. **Synthesizer (AI Research Analyst):** Rather than summarizing isolated papers, this agent acts as an analytical synthesizer—detecting points of consensus, theoretical debates, trade-offs, and research gaps across retrieved sources.
4. **Report Writer (Lead Academic Technical Writer):** Specializes exclusively in narrative structure, academic tone, LaTeX equation rendering, and citation formatting in Markdown.

Design Trade-offs & Process Selection

- **Sequential vs. Hierarchical Crew:** This example chose `Process.sequential` over a Hierarchical architecture with a manager agent. I see that this type of review has a natural linear dependency: You can't write a structured response to a single question without sub-questions, cannot synthesize without sources, and cannot write without synthesis. A manager agent would have introduced unnecessary token latency, cost, and decision overhead.
 - **Explicit Context Chaining vs. RAG Memory:** Instead of relying on CrewAI's automatic memory system, we used explicit task context dependencies (`context=[expand_task, search_task, ...]`). This guaranteed that downstream agents received 100% full-text fidelity of previous findings rather than lossy, truncated memory chunks.
-

2. The ReAct Paradigm in Action

The execution traces demonstrated the **ReAct (Reasoning + Acting)** pattern, specifically within the **Source Hunter** agent. Rather than performing a single naive vector search, the agent engaged in an iterative **Thought** → **Action** → **Observation** loop:

- **Thought:** *"I need to discover evidence on how multi-agent systems coordinate and prevent communicative deadlocks. First, I should search for CAMEL and communication protocols."*

- **Action:** `search_papers(query="CAMEL communicative agents role playing inception prompting", k=5)`
- **Observation:** The tool returned relevant excerpts from `08_camel.pdf` explaining inception prompting and role flipping.
- **Thought:** *"The CAMEL paper provides cooperative dialogue evidence, but I still lack evidence on how conversational frameworks handle human-in-the-loop oversight and code execution. I need to search AutoGen next."*
- **Action:** `search_papers(query="AutoGen multi-agent conversation UserProxyAgent execution", k=5)`
- **Observation:** The tool returned passages from `10_autogen.pdf` highlighting `UserProxyAgent` safety modes and Docker sandboxing.

This interleaved loop prevented open-loop hallucination: each subsequent search query was grounded in what the agent had already observed, mimicking how human researchers systematically navigate literature.

3. Debugging Journey & Key Technical Learnings

Building and debugging this pipeline provided some of the most valuable learnings of the assignment:

1. Embedding Dimension Mismatch (3072 vs. 768):

The initial verification script failed during embedding validation. The starter kit hardcoded Gemini embeddings to 768 dimensions (the legacy standard), whereas Google's modern `gemini-embedding-001` defaults to **3072 dimensions** via Matryoshka Representation Learning. Because ChromaDB was already indexed with 3072-dimensional vectors from the PDFs, aligning `dimension = 3072` in `tools/embeddings.py` resolved the mismatch and enabled high-precision cosine similarity retrieval (scores > 0.88).

2. Model Lifecycle & Modern SDK Routing:

The starter kit used deprecated configurations for `gemini-2.0-flash` which returned a 404 from Google's API. Updating to `gemini-3.8-flash` in `.env` solved the model availability. Furthermore, modern CrewAI automatically detected the `gemini/` prefix and routed calls natively to Google's official `google-genai` SDK rather than relying on legacy OpenAI translation proxies, requiring the installation of `crewai[google-genai]`.

3. Shell Environment Persistence vs. `load_dotenv`:

When updating `.env` to `gemini-3.8-flash`, the application initially continued calling the retired model. I discovered that Python's `load_dotenv()` defaults to `override=False`, meaning pre-existing environment variables in the active terminal session took precedence over changes written to disk. Explicitly setting `load_dotenv(override=True)` and defining `llm="gemini/gemini-3.8-flash"` on agent definitions established deterministic configuration.

4. Disabling Built-in Memory (`memory=False`):

Enabling `memory=True` in CrewAI triggered unexpected HTTP 400 errors because CrewAI's internal memory storage hardcodes OpenAI embedding endpoints. Disabling this eliminated errors while preserving complete context via CrewAI's native task `context` chaining.

5. Removing outputs from `.gitignore` Starter Kit Gitignore Trap:

The starter kit had `outputs/*` defined in `.gitignore`, which prevented generated reports from being tracked by git. Removing `outputs/*` from `.gitignore` was necessary to ensure the reports were committed and pushed for evaluation.

4. Production Considerations

To evolve this prototype into an enterprise-grade production research platform, several enhancements would be required:

1. **Dynamic & Multi-Modal Ingestion:** Extending the RAG pipeline beyond local static PDFs to include live arXiv API search, Semantic Scholar web-scraping agents, and web retrieval tools to continuously update the corpus.
2. **Asynchronous Parallel Retrieval:** The current sequential execution runs tool queries one by one. In production, the **Source Hunter** should execute multi-query searches in parallel using asynchronous I/O (**asyncio**) to reduce latency from minutes to seconds.
3. **Automated Citation & Fact Verification:** Incorporating an automated critic agent running formal NLI (Natural Language Inference) to verify that every claim in the final review is strictly entailed by the referenced chunk ID before saving the report.
4. **Token Economics & Rate Limiting:** Implementing token budgets, caching frequent queries, and adding exponential backoff retries to prevent hitting API 429 quota thresholds.
5. **Streaming & Observability:** Integrating OpenTelemetry tracing and streaming LLM token responses to a user-facing dashboard for real-time visibility into agent reasoning steps.